

COMP 3000 (WINTER 2023) OPERATING SYSTEMS

TUTORIAL 8 [\[LINK\]](#)

Tasks part A: Getting Started

Compile `3000physicalview` and `3000memview2` using the provided Makefile (i.e., by running `make`).

Insert `3000physicalview` by running `make insert`. Confirm that the module is inserted using `lsmod`.

1. Examine the call to `copy_from_user()` and `copy_to_user()` on lines 120 and 132 of `3000physicalview.c` (as also discussed in the lecture). Consider the following:
 - How are these functions different from `get_user()/put_user()` that we have seen in the previous tutorial?
`copy_from_user()/copy_to_user()` is comparable to `memcpy()` which deals with a range of memory, whereas `get_user()/put_user()` is used for single variables (e.g., a char or int).
 - Why are these functions necessary? Couldn't we just access the userspace address directly? What could happen if we did?
If you try to use `memcpy()` to replace these four functions it *may* work. However, losing fault containment as part of the price for running code in kernel space, any invalid addresses passed into the kernel mode (e.g., null, or pointers to other processes or even the kernel's memory) will lead to kernel oops or even panic. These four functions do all the access checks and ensure that the addresses are valid.
2. `3000physicalview` exposes its API to userspace in the form of an `ioctl(2)` call. Consider the following:
 - What is an `ioctl`? How is it different from a read or write system call? Hint: check `man 2 ioctl`.
In addition to the differences in syntax, consider `ioctl()` and `read/write()` in two aspects: `read` and `write` are natural operations to a file, also applicable to special files. If they are called in-band, we can call `ioctl` an out-of-band channel, which can adjust the behavior of regular reads/writes. On the other hand, `read()` is one-way only for reading and `write()` is one-way only for writing. `ioctl()` is two-way: you can provide input and get output to fulfill any tasks.
 - How does `3000physicalview` implement its `ioctl`? What arguments does it take?
In `physicalview_ioctl()` in `3000physicalview.c`, `ioctl` takes a file descriptor (referring to the device), a command (`PHYSICALVIEW_WALK`), and an address pointing to a custom data structure (one member being the virtual address and the other being the returned physical address).
 - How does `3000memview2` call the `ioctl`? What arguments does it pass to the `ioctl`?
Looking at the call to `ioctl()` in `3000memview2.c`, the arguments passed in correspond to the three above. In particular, `PHYSICALVIEW_WALK` is defined in `3000physicalview.h`. For now, just treat it as a value agreed on by both the driver and the application to perform this translation.

3. Which function does the virtual-to-physical translation? What type is `current->mm`? How does it give you the address of the `pgd`? Recall the page table walk explained in the lecture and see how it's reflected in this function (writing it down is optional).

`get_physical()`.

`struct mm_struct`. Here you see how page tables can be per-process.

`mm_struct` contains a member pointer: `pgd_t * pgd`, pointing to its `pgd`.

Each `pgd_offset()` takes the address of its parent-level table, and returns an address to an `pte` entry corresponding to a given virtual address, all the way down to a `pte`.

4. Once you've got the page frame number (pfn), how is the physical address (phys) calculated? Describe in words.

The page frame number is retrieved from the `pte` in the last-level page table, referring to a physical page frame but not the page frame address yet. For example, if the number is `0x1234`, the address should be `0x1234000` (as the page size is 4KiB, taking 12 bits = 3 nibbles). Therefore, to get the translated physical address, we use the page frame start address (`0x1234` shifted to the left by 12 bits = `0x1234000`) + the offset within the page frame (extracting the last 12 bits from `addr`).

Tasks part B: Examining Physical Memory Mappings

1. With `3000physicalview` inserted, run `3000memview2` and examine the output. Note that it presents virtual memory addresses on the left, and physical addresses on the right. Are these mappings consistent with what you expected (e.g., in terms of patterns)?

The virtual addresses on the left demonstrates the layout of a typical memory space (as mentioned in Tutorial 2 and corresponding lectures), while the physical addresses on the right show no patterns overall.

2. Compare `3000memview2` with `3000memview` from Tutorial 2. What is similar about their code, and what is different? How similar is their output?

They both define various types of variables and print their addresses. `3000memview` uses `printf()` on their virtual addresses while `3000memview2` uses the `3000physicalview` driver to report their physical addresses as well as the virtual ones.

3. Do you notice a pattern in the virtual addresses of `buf[i]`? Is this same pattern present in the physical addresses? Why or why not?

NOTE: please use [this updated 3000memview2.c](#) to be able to see the observations discussed in class and explained below:

If you look at the last three nibbles (e.g., `6b0`) of the addresses of `buf[i]` (in the code `buf[0]` is printed), you'll see they are incrementing (continuous) and the same is seen in both virtual and physical addresses. This shows that memory is managed (allocated) in pages. No matter how the mapping is done (maybe not as you expected), the granularity is page.

The original code was printing the first byte of `buf[0]`, `buf[1]`, `buf[2]`, ... `buf[i]` which were individually allocated from `malloc()`, each being 4K. So, there addresses are not supposed to be continuous. The updated code at the end prints the first 30 bytes of `buf[0]` within its 4K.

4. Run `3000memview2` a few more times and consider the following:

- Are the virtual addresses the same or different between runs? How about physical addresses?

In general, all (both virtual and physical) addresses are changing. ASLR contributes to the change of virtual addresses, and the kernel just allocates/deallocates memory pages affected by many factors not pertaining to the process in question hence the

change of physical addresses.

- Some physical addresses don't seem to be changing between runs. Which ones? Why do you think this might be the case?
main, lmsg and gmsg. See below.

5. Force the kernel to drop the virtual memory cache using `sync && echo 3 | sudo tee /proc/sys/vm/drop_caches`. Run `3000memview2` one more time and note that the physical addresses that stayed the same previously have now changed. What do you think just happened?

What is cached and what is not:

Recall that the page cache is actually “disk cache” which stores recently accessed data from block devices, including files. Then it’s about which part of the address space is from files and which is not. Checking the memory layout (e.g., using `pmap`) you can see the program binary and various dynamic libraries are from files, while the rest is not, e.g., stack, heap, bss, etc. Among the variables of `3000memview2`, only `lmsg` and `gmsg` (in the data segment) and `main` (in the text/code segment) are from the program binary file `3000memview2`, hence cached.

Tasks part C: Using `bpftool` to Monitor `3000physicalview` and `3000memview2`

Now recall what you tried with eBPF in Tutorial 7. We can also do something similar to watch the interaction between `3000physicalview` and `3000memview2`. Note: No in-depth understanding of eBPF is expected in this course. You only need to understand what is involved and discussed. Feel free to read more if interested.

1. Use the following “one-liner” (as it is just a one-line string within the single quotation marks) in a new terminal session first and in the original one run `./3000memview2` as above:

```
sudo bpftool trace -e 'tracepoint:syscalls:sys_enter_ioctl { printf("%s: fd=%d; cmd=%d; arg=%ld \n", comm, args->fd, args->cmd, args->arg); }' | grep 3000memview2
```

What does this one-liner do? As always, you can check `man bpftool` if needed.

It hooks to an event called “`tracepoint:syscalls:sys_enter_ioctl`” which corresponds to the time when entering the `ioctl()` system call (there’s also one corresponding to its exit --- “`tracepoint:syscalls:sys_exit_ioctl`”). Then it prints information for its arguments, i.e., the involved file descriptor, command, and argument (in our case it’s the pointer to that C struct).

2. (Optional in the submission but you should do it) It seems the command above does not show useful information as the third argument is actually a pointer. So, to be able to show the two arguments `virt` and `phys` defined in `3000physicalview.h`, you will need to create a *.bt file. Refer to `/usr/sbin/*.bt` and convert the one-liner above into something like `snoop3000physicalview.bt`, so that when it runs it prints the `virt` passed in and the `phys` that is returned. The code will be posted and explained.

Note: if you choose to directly include the header file you need to create a copy and remove the `MODULE_*` macros. Otherwise, you can simply include the struct definition.

See the code at the end.

As it’s just the C syntax, you can include the C struct definition from the header file and dereference `arg` to print the two members. Note here that upon entry there’s only the virtual address passed in so you can only print the physical address upon return. But you first need to store the argument (pointer) in an eBPF map (a variable whose name starts with an “@”) as it’s unavailable in the exit event. Also, don’t forget to free up memory using `clear()`.

```

#!/usr/bin/env bpftrace
struct physicalview_memory
{
    unsigned long virt;
    unsigned long phys;
};
/* Optionally, we can add a BEGIN section to print some messages */
BEGIN
{
    printf("Tracing ioctl calls from all processes... Hit Ctrl-C to end.\n");
}
tracepoint:syscalls:sys_enter_ioctl
{
    if (comm == "3000memview2")
    {
        @param[tid] = (struct physicalview_memory *) (args->arg);
        printf("%s: fd=%d; cmd=0x%x; virt=0x%016llx, phys=0x%016llx\n",
comm, args->fd, args->cmd, ((struct physicalview_memory *)args->arg)->virt,
((struct physicalview_memory *)args->arg)->phys);
    }
}
tracepoint:syscalls:sys_exit_ioctl
{
    if (comm == "3000memview2")
    {
        printf("%s: virt=0x%016llx, phys=0x%016llx\n\n", comm,
@param[tid]->virt, @param[tid]->phys);
    }
}
END
{
    clear(@param);
}

```